Problem Statement — Blog Platform REST API Backend

Backend Assignment

A growing blogging platform wants to build a secure and scalable backend system to manage blog publishing, user roles, moderation workflows, comments, and platform administration.

Currently, the platform does not have a proper backend application to handle user authentication, blog approvals, content moderation, or role-based access control. The management team wants a complete REST API backend that can support multiple types of users and real-world publishing workflows.

As a backend developer, your responsibility is to design and develop a production-ready REST API backend using Node.js, Express.js, and MongoDB.

The system must support three different user roles:

- Author
- Moderator
- Admin

Each role should have different permissions and responsibilities within the platform.

System Requirements

Author Responsibilities

Authors should be able to:

- Register and log in securely
- Create blog posts
- Save posts as drafts
- Edit or delete their own posts
- Submit posts for moderation review
- Add cover image URLs and tags
- Reply to comments on their posts
- View analytics related to their posts
- Manage their own profile

Authors should NOT be able to publish posts directly.

Moderator Responsibilities

Moderators are responsible for reviewing content submitted by authors.

Moderators should be able to:

- View all pending posts
- Approve posts for publishing
- Reject posts with proper feedback
- Hide or delete inappropriate comments
- Flag suspicious posts for admin review
- Handle spam reports

Moderators should NOT be able to create blog posts.

Admin Responsibilities

Admins should have complete control over the platform.

Admins should be able to:

- Manage all users
- Ban or unban users
- Change user roles
- Delete any post or comment

```
    // controller
    if(req.role == "admin") {
        Post.findByIdAndDelete({ _id: id })
    } else {
        Post.findOneAndDelete({ _id: id, userId: req.userId })
    }
```

- Create and manage categories
- View overall platform analytics
- Review flagged content

Admins inherit both Author and Moderator permissions.

// app.post('/api/blogs', authenticateuser, authorizeUser(['author', 'admin']), blogsCltr.create); // app.post('/api/blogs/peding', authenticateuser, authorizeUser(['moderator', 'admin']), blogsCltr.pending);

Blog Post Workflow

The application must support the following publishing workflow:

```
draft → pending → published
              ↓
          rejected
```

Workflow Explanation:

- Authors create blog posts as drafts
- Draft posts are submitted for review
- Moderators/Admins approve or reject posts
- Rejected posts must contain feedback
- Authors can edit rejected posts and resubmit them

Technical Requirements

Students must build the project using:

- Node.js
- Express.js
- MongoDB Local Database
- Mongoose
- JWT Authentication
- bcrypt
- dotenv

Database Requirement

Students must install MongoDB locally on their machine and use a local MongoDB connection.

Example connection string:

```
MONGO_URI=mongodb://127.0.0.1:27017/blog_platform
```

Required Features

The backend application must include:

- User Authentication
- JWT Authorization
- Role-Based Access Control
- Blog Post Management
- Moderation Workflow
- Nested Comments
- Categories Management
- User Management
- Search Functionality
- Pagination
- Filtering

- Analytics APIs
- Centralized Error Handling

Required Mongoose Schemas

Students must create the following Mongoose schemas using proper field types, validations, references, enums, default values, and timestamps.

The schemas should demonstrate proper database design and relationships between collections.

---

1. User Schema

The User schema stores all platform users including Authors, Moderators, and Admins.

Required Fields

Field Name	Type	Requirements
name	String	Required
email	String	Required, Unique, Lowercase
password	String	Required, Hashed
role	String	Enum: `author`, `moderator`, `admin`
bio	String	Optional
avatar	String	Optional
isBanned	Boolean	Default: `false`
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

---

2. Post Schema

The Post schema stores all blog posts created by authors.

Required Fields

Field Name	Type	Requirements
title	String	Required
slug	String	Required, Unique

Field Name	Type	Requirements
content	String	Required
status	String	Enum: `draft`, `pending`, `published`, `rejected`
author	ObjectId	Reference to User
category	ObjectId	Reference to Category
tags	Array of Strings	Optional
coverImage	String	Optional
views	Number	Default: `0`
likes	Array of ObjectIds	Reference to User
rejectionReason	String	Optional
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

---

3. Comment Schema

The Comment schema stores comments and threaded replies for blog posts.

Required Fields

Field Name	Type	Requirements
body	String	Required
author	ObjectId	Reference to User
post	ObjectId	Reference to Post
parentComment	ObjectId	Reference to Comment
isHidden	Boolean	Default: `false`
flaggedBy	Array of ObjectIds	Reference to User
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

---

4. Category Schema

The Category schema stores blog categories.

Required Fields

Field Name	Type	Requirements
name	String	Required, Unique
slug	String	Required, Unique
description	String	Optional
createdBy	ObjectId	Reference to User
isActive	Boolean	Default: `true`
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

Schema Relationship Requirements

Students must properly establish relationships between collections using `ObjectId` references.

Required Relationships

Relationship	Description
Post → User	Each post belongs to one author
Post → Category	Each post belongs to one category
Comment → User	Each comment belongs to one user
Comment → Post	Each comment belongs to one post
Comment → Comment	Support nested replies
Category → User	Category created by admin

Important Mongoose Concepts Students Must Use

Students are expected to properly use:

- `type`
- `required`
- `default`
- `enum`
- `unique`
- `trim`
- `lowercase`
- `timestamps`
- `ObjectId`
- `ref`

- Arrays
- Schema relationships
- Nested references

API Requirements

Students must implement REST APIs for:

- Authentication
- Posts
- Moderation
- Comments
- Categories
- Admin Operations
- Analytics

All APIs must follow proper REST standards and return consistent JSON responses.

Architecture Requirements

Students must follow proper MVC architecture.

Suggested folder structure:

```
project-root/
│
├── controllers/
├── models/
├── routes/
├── middleware/
├── utils/
├── config/
├── validators/
├── services/
│
├── app.js
├── server.js
└── package.json
```

Security Requirements

The application must ensure:

- Passwords are hashed using bcrypt
- JWT authentication is implemented securely
- Protected routes use middleware
- Role checks are handled properly
- Banned users cannot access protected APIs
- Environment variables are stored in `.env`

Deliverables

Students must submit:

1. GitHub Repository

- Public repository
- Proper folder structure
- Minimum 10 meaningful commits

---

2. README.md

The README must include:

- Project overview
- Local MongoDB setup steps
- Installation instructions
- Environment variables
- API documentation

---

3. Postman Collection

Students must provide:

- All API endpoints
- Sample request bodies
- Authentication token testing

Evaluation Criteria

Students will be evaluated based on:

Code Quality

- Clean architecture
- Reusable code
- Proper async/await usage

- Organized project structure

Security

- JWT implementation
- Password hashing
- Authorization checks

API Design

- Proper REST API standards
- Correct HTTP status codes
- Validation handling
- Consistent response format

Git Practices

- Meaningful commits
- Proper `.gitignore`
- No `node_modules`
- No `.env` file in repository

Bonus Features (Optional)

Students can implement the following for extra credit:

- Image upload using Cloudinary or Multer
- Email notifications using Nodemailer
- Refresh token authentication
- Rate limiting
- Full-text search
- Unit testing using Jest + Supertest

Objective of the Assignment

The purpose of this assignment is to help students understand how real-world backend systems are designed and developed using modern backend technologies.

Students are expected to focus on:

- Clean code practices
- API design
- Authentication & authorization

- Database relationships
- Security
- Scalable backend architecture
- Professional project structure

Complete API List — Blog Platform REST API Backend

Module	Method	Endpoint	Access	Description
Auth	POST	`/api/auth/register`	Public	Register a new user
Auth	POST	`/api/auth/login`	Public	Login user and return JWT
Auth	GET	`/api/auth/me`	Authenticated	Get current user profile
Auth	PUT	`/api/auth/me`	Authenticated	Update current user profile

Posts APIs

Module	Method	Endpoint	Access	Description
Posts	POST	`/api/posts`	Author/Admin	Create new post as draft
Posts	GET	`/api/posts`	Public	Get all published posts
Posts	GET	`/api/posts/:slug`	Public	Get single post by slug
Posts	PUT	`/api/posts/:id`	Owner/Admin	Update own draft/rejected post
Posts	DELETE	`/api/posts/:id`	Owner/Admin	Delete post
Posts	PATCH	`/api/posts/:id/submit`	Owner	Submit draft for review
Posts	PATCH	`/api/posts/:id/like`	Authenticated	Like/unlike post
Posts	GET	`/api/posts/my`	Author/Admin	Get logged-in user's posts

Moderation APIs

Module	Method	Endpoint	Access	Description
Moderation	GET	`/api/moderation/pending`	Moderator/Admin	Get all pending posts
Moderation	PATCH	`/api/moderation/:id/approve`	Moderator/Admin	Approve post
Moderation	PATCH	`/api/moderation/:id/reject`	Moderator/Admin	Reject post with feedback

Module	Method	Endpoint	Access	Description
Moderation	PATCH	`/api/moderation/:id/flag`	Moderator/Admin	Flag suspicious post

Comments APIs

Module	Method	Endpoint	Access	Description
Comments	GET	`/api/posts/:id/comments`	Public	Get all comments for a post
Comments	POST	`/api/posts/:id/comments`	Authenticated	Add comment or reply
Comments	DELETE	`/api/comments/:id`	Owner/Moderator/Admin	Delete comment
Comments	PATCH	`/api/comments/:id/hide`	Moderator/Admin	Hide/unhide comment
Comments	PATCH	`/api/comments/:id/flag`	Authenticated	Flag comment

Categories APIs

Module	Method	Endpoint	Access	Description
Categories	GET	`/api/categories`	Public	Get all categories
Categories	POST	`/api/categories`	Admin	Create category
Categories	PUT	`/api/categories/:id`	Admin	Update category
Categories	DELETE	`/api/categories/:id`	Admin	Delete category

Admin APIs

Module	Method	Endpoint	Access	Description
Admin	GET	`/api/admin/users`	Admin	Get all users
Admin	PATCH	`/api/admin/users/:id/role`	Admin	Change user role
Admin	PATCH	`/api/admin/users/:id/ban`	Admin	Ban/unban user
Admin	GET	`/api/admin/stats`	Admin	Get platform statistics

Optional Analytics APIs

Module	Method	Endpoint	Access	Description
Analytics	GET	`/api/analytics/posts/:id`	Owner/Admin	Get analytics for a post
Analytics	GET	`/api/analytics/my-posts`	Author/Admin	Get analytics for own posts

Query Parameters Support

Feature	Example
Pagination	`/api/posts?page=1&limit=10`
Search	`/api/posts?q=nodejs`
Filtering	`/api/posts?category=javascript`
Sorting	`/api/posts?sort=latest`

Post Status Workflow APIs

Workflow Step	Endpoint
Draft Creation	`POST /api/posts`
Submit for Review	`PATCH /api/posts/:id/submit`
Approve Post	`PATCH /api/moderation/:id/approve`
Reject Post	`PATCH /api/moderation/:id/reject`

What is a Slug?

A **slug** is a URL-friendly version of a title or text.

It is mainly used in web applications to create clean, readable, and SEO-friendly URLs.

Example

Blog Title

```
How to Learn Node.js Fast
```

Generated Slug

```
how-to-learn-nodejs-fast
```

Why Do We Use Slugs?

Instead of using database IDs in URLs like this:

```
/api/posts/6823acb2213fd9
```

We use readable URLs:

```
/api/posts/how-to-learn-nodejs-fast
```

Advantages of Slugs

Benefit	Explanation
Readable URLs	Easier for users to understand
SEO Friendly	Better for search engines
Cleaner APIs	More professional URLs
Easier Sharing	Users can recognize content from URL

How Slug Works in This Project

When an author creates a post:

```
{
  "title": "Introduction to Express JS"
}
```

The backend automatically generates:

```
introduction-to-express-js
```

This slug gets stored in the database.

Database Example

Post Document

```
{
  "_id": "6823abc123",
  "title": "Introduction to Express JS",
  "slug": "introduction-to-express-js",
  "content": "..."
}
```

API Usage with Slug

Endpoint

```
GET /api/posts/introduction-to-express-js
```

Instead of:

```
GET /api/posts/6823abc123
```

How to Generate Slug in Node.js

Students can use the `slugify` package.

Install Package

```
npm install slugify
```

Example Usage

```
const slugify = require("slugify");

const title = "Introduction to Express JS";

const slug = slugify(title, {
  lower: true,
  strict: true
});

console.log(slug);
```

Output

```
introduction-to-express-js
```

Slug Generation Options

Option	Purpose
lower: true	Converts to lowercase
strict: true	Removes special characters

Real Example

Input Title

```
Node.js & Express Crash Course!!!
```

Generated Slug

```
nodejs-express-crash-course
```

Where Slug Should Be Generated

Slug should usually be generated:

- During post creation
- During post title update

Example in Controller

```
const slugify = require("slugify");

const slug = slugify(req.body.title, {
  lower: true,
  strict: true
});

const post = await Post.create({
  title: req.body.title,
  slug,
  content: req.body.content
});
```

Important Problem — Duplicate Slugs

What if two posts have the same title?

Example:

```
How to Learn React
```

Another user creates the same title.

Both generate:

```
how-to-learn-react
```

This causes duplicate slug issues.

Solution for Duplicate Slugs

Students can append:

- timestamp
- random number
- MongoDB ObjectId

Example

```
how-to-learn-react-6823
```

Recommended Development Order

Students should proceed in the following order while building the project.

This order is important because:

- Posts depend on Categories
- Posts reference Category ObjectIds
- Categories must exist before creating posts

Recommended Order

```
1. Project Setup
2. MongoDB Connection
3. Authentication System
4. JWT & Role Middleware
5. Categories Module
6. Posts CRUD Module
7. Moderation Workflow
8. Comments Module
9. Admin Module
10. Search, Pagination & Filtering
11. Error Handling & Validation
12. Testing Using Postman
13. GitHub & Documentation
```

Step-by-Step Development Flow

Phase 1 — Project Setup

Step 1 — Create Project Folder

```
mkdir blog-platform-api
cd blog-platform-api
```

Step 2 — Initialize Node Project

```
npm init -y
```

Step 3 — Install Dependencies

```
npm install express mongoose bcrypt jsonwebtoken dotenv cors slugify
```

Step 4 — Install Development Dependencies

```
npm install --save-dev nodemon
```

Step 5 — Create Folder Structure

```
project-root/
│
├── config/
├── controllers/
├── middleware/
├── models/
├── routes/
├── validators/
├── services/
├── utils/
│
├── app.js
├── server.js
├── .env
└── package.json
```

Step 6 — Configure package.json Scripts

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

Phase 2 — MongoDB Connection

Step 7 — Install MongoDB Community Server

Students must:

- Install MongoDB locally
- Start MongoDB service

Step 8 — Configure Environment Variables

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/blog_platform
JWT_SECRET=mysecretkey
```

Step 9 — Create Database Connection

File

```
config/db.js
```

Step 10 — Start Express Server

Responsibilities

- Load dotenv
- Connect MongoDB
- Start server

Phase 3 — Authentication System

Step 11 — Create User Schema

File

```
models/User.js
```

Step 12 — Create Register API

Endpoint

```
POST /api/auth/register
```

Step 13 — Hash Password Using bcrypt

```
const hashedPassword = await bcrypt.hash(password, 10);
```

Step 14 — Create Login API

Endpoint

```
POST /api/auth/login
```

Step 15 — Generate JWT Token

```
jwt.sign(
  { userId: user._id },
  process.env.JWT_SECRET,
  { expiresIn: "7d" }
);
```

Phase 4 — JWT & Role Middleware

Step 16 — Create JWT Middleware

File

```
middleware/authMiddleware.js
```

Step 17 — Create Role Middleware

File

```
middleware/roleMiddleware.js
```

Responsibilities

```
requireRole("admin")
```

Phase 5 — Categories Module

Step 18 — Create Category Schema

File

```
models/Category.js
```

Step 19 — Create Category API

Endpoint

```
POST /api/categories
```

Step 20 — Get Categories API

Endpoint

```
GET /api/categories
```

Step 21 — Update Category API

Endpoint

```
PUT /api/categories/:id
```

Step 22 — Delete Category API

Endpoint

```
DELETE /api/categories/:id
```

Phase 6 — Posts CRUD Module

Step 23 — Create Post Schema

File

```
models/Post.js
```

Step 24 — Install slugify

```
npm install slugify
```

Step 25 — Generate Slug

```
const slug = slugify(title, {
  lower: true,
  strict: true
});
```

Step 26 — Create Post API

Endpoint

```
POST /api/posts
```

Functionalities

- Create draft
- Add category reference
- Add tags
- Add cover image
- Save slug

Step 27 — Get All Posts API

Endpoint

```
GET /api/posts
```

Functionalities

- Pagination
- Search
- Filtering
- Sorting

Step 28 — Get Single Post API

Endpoint

```
GET /api/posts/:slug
```

Step 29 — Update Post API

Endpoint

```
PUT /api/posts/:id
```

Step 30 — Delete Post API

Endpoint

```
DELETE /api/posts/:id
```

Phase 7 — Moderation Workflow

Step 31 — Submit Post for Review

Endpoint

```
PATCH /api/posts/:id/submit
```

Workflow

```
Draft → Pending
```

Step 32 — Get Pending Posts

Endpoint

```
GET /api/moderation/pending
```

Step 33 — Approve Post

Endpoint

```
PATCH /api/moderation/:id/approve
```

Workflow

```
Pending → Published
```

Step 34 — Reject Post

Endpoint

```
PATCH /api/moderation/:id/reject
```

Responsibilities

- Store rejection reason

Phase 8 — Comments Module

Step 35 — Create Comment Schema

File

```
models/Comment.js
```

Step 36 — Add Comment API

Endpoint

```
POST /api/posts/:id/comments
```

Step 37 — Get Comments API

Endpoint

```
GET /api/posts/:id/comments
```

Step 38 — Delete Comment API

Endpoint

```
DELETE /api/comments/:id
```

Step 39 — Hide Comment API

Endpoint

```
PATCH /api/comments/:id/hide
```

Phase 9 — Admin Module

Step 40 — Get Users API

Endpoint

```
GET /api/admin/users
```

Step 41 — Change User Role API

Endpoint

```
PATCH /api/admin/users/:id/role
```

Step 42 — Ban User API

Endpoint

```
PATCH /api/admin/users/:id/ban
```

Step 43 — Platform Statistics API

Endpoint

```
GET /api/admin/stats
```

Phase 10 — Search, Pagination & Filtering

Step 44 — Add Pagination

```
/api/posts?page=1&limit=10
```

Step 45 — Add Search

```
/api/posts?q=nodejs
```

Step 46 — Add Filtering

```
/api/posts?category=javascript
```

Phase 11 — Error Handling & Validation

Step 47 — Create Error Middleware

Responsibilities

- JWT errors
- MongoDB errors
- Validation errors
- Status codes

Step 48 — Add Request Validation

```
npm install express-validator
```

Phase 12 — Testing Using Postman

Step 49 — Test All APIs

Students must:

- Test protected routes
- Test authorization

- Test workflows
- Save Postman collection

Phase 13 — GitHub & Documentation

Step 50 — Initialize Git

```
git init
```

Step 51 — Create .gitignore

```
node_modules
.env
```

Step 52 — Push Project to GitHub

Students must:

- Create meaningful commits
- Maintain clean structure
- Add proper README
- Push final project repository